Trader-Company Method: A Metaheuristic for Interpretable Stock Price Prediction

Katsuya Ito
Preferred Networks, Inc.
katsuya1ito@preferred.jp

Kentaro Minami
Preferred Networks, Inc.
minami@preferred.jp

Kentaro Imajo
Preferred Networks, Inc.
imos@preferred.jp

Kei Nakagawa
Nomura Asset Management Co., Ltd.
k-nakagawa@nomura-am.co.jp

ABSTRACT

Investors try to predict returns of financial assets to make successful investment. Many quantitative analysts have used machine learning-based methods to find unknown profitable market rules from large amounts of market data. However, there are several challenges in financial markets hindering practical applications of machine learning-based models. First, in financial markets, there is no single model that can consistently make accurate prediction because traders in markets quickly adapt to newly available information. Instead, there are a number of ephemeral and partially correct models called "alpha factors". Second, since financial markets are highly uncertain, ensuring interpretability of prediction models is quite important to make reliable trading strategies. To overcome these challenges, we propose the Trader-Company method, a novel evolutionary model that mimics the roles of a financial institute and traders belonging to it. Our method predicts future stock returns by aggregating suggestions from multiple weak learners called Traders. A Trader holds a collection of simple mathematical formulae, each of which represents a candidate of an alpha factor and would be interpretable for real-world investors. The aggregation algorithm, called a Company, maintains multiple Traders. By randomly generating new Traders and retraining them, Companies can efficiently find financially meaningful formulae whilst avoiding overfitting to a transient state of the market. We show the effectiveness of our method by conducting experiments on real market data.

KEYWORDS

Finance, Metaheuristics, Stock Price Prediction

ACM Reference Format:
Katsuya Ito, Kentaro Minami, Kentaro Imajo, and Kei Nakagawa. 2021. Trader-Company Method: A Metaheuristic for Interpretable Stock Price Prediction. In *Proc. of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2021), Online, May 3–7, 2021*, IFAAMAS, 9 pages.

1 INTRODUCTION

Developing quantitative trading strategies is a universal task in the financial industry [11]. Many quantitative models have been proposed to predict the behavior of financial markets [20, 32, 41]. For example, Fama–Frech's three-factor model and five-factor model [7, 15, 16] have been standard asset pricing models for many years. Technical indicators such as Moving Average Convergence Divergence (MACD) and Relative Strength Index (RSI) are also prediction methods that have been used by many traders [1, 28].

Although many quantitative analysts are struggling to derive new rules from newly available big data, there has been no gold-standard practical method that can fully leverage these data [41]. We believe that there are the following two challenges that are hindering the development of quantitative models today.

1.1 Tackling Nearly-Efficient Markets

Our first challenge is to tackle the non-stationary and noisy nature of the financial market, which is known as the *efficiency* of markets. The widely acknowledged Efficient Market Hypothesis [14] states that asset prices reflect all available information, correcting undervalued or overvalued prices into fair values. In an efficient market, investors cannot outperform the overall market because asset prices quickly follow other traders' strategic and adversarial activities [10]. In fact, many empirical studies have reported that real-world markets are nearly efficient [9, 32, 41]. Due to this, future stock returns are hardly predictable in most markets, and no single explanatory model can consistently make an accurate prediction.

On the other hand, there still is a common belief that stock returns can be predictable in a sufficiently short time period, which suggests the existence of investments or trading strategies that beat the overall market at least temporarily. In particular, potential sources of profitability would come from some simple mathematical formulae, called *alpha factors* [13, 26]. Typical alpha factors used in production are given as combinations of few elementary functions and arithmetic operations. For example,

$$\log(\text{yesterday's close price}/\text{yesterday's open price})$$

represents the classical momentum strategy [24]. It has been reported that there is a variety of mathematical formulae with reasonably low mutual correlations [26], each of which can be a good trading signal and actually usable in real-life trading.

Although the efficacy of a single formula is slight and ephemeral, combining multiple formulae in a sophisticated way can lead to a more robust trading signal. We hypothesize that we can overcome the instability and the uncertainty of markets by maintaining multiple "weak models" given as simple mathematical formulae. This is in the same spirit as the ensemble methods (see e.g., [21, 39]),

Proc. of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2021), U. Endriss, A. Nowé, F. Dignum, A. Lomuscio (eds.), May 3–7, 2021, Online.

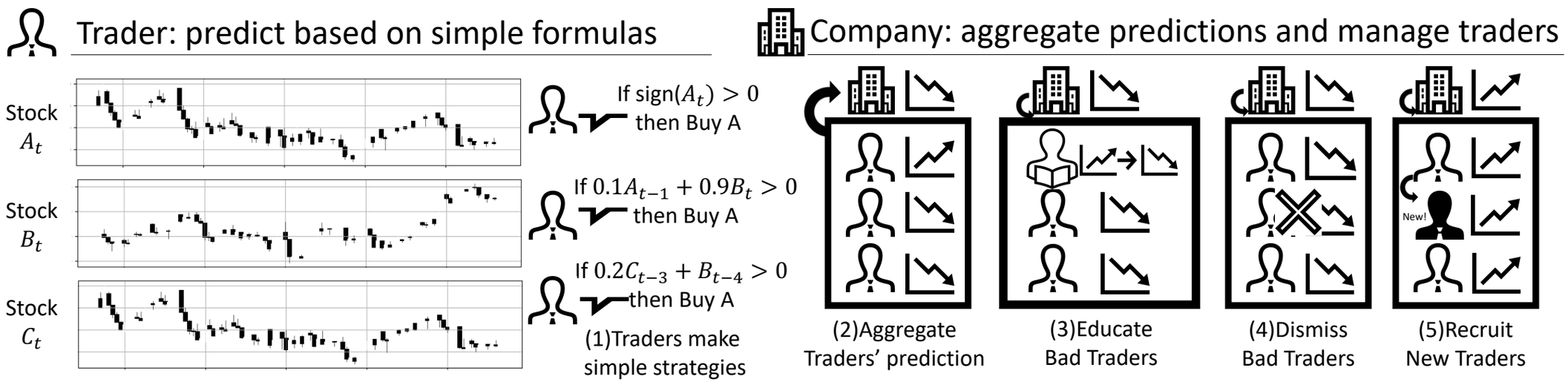

Figure 1: Illustration of our method. (1) Traders predict the return of assets using simple formulas. Companies manage Traders and combine Traders' predictions into one value. The Company algorithm consists of four functions: (2) prediction by aggregation, (3) education of bad Traders, (4) dismissal of bad Traders, and (5) recruitment of new Traders.

but paying more attention to specific structure of real-world alpha factors may improve the performance of the resulting trading strategy.

1.2 Interpretability of Trading Strategies

The second challenge is to gain the interpretability of models. As mentioned above, it is hard to achieve consistently high performance in the financial markets. Even if we could have the best possible trading strategies at hand, their predictive accuracies are quite limited and unsustainable. To gain intuition, suppose that we forecast the rise or fall of a single stock. Then, the accuracy is typically no higher than 51%, which is approximately the chance rate. As such, investors should worry about their trading strategies having a large uncertainty in the returns and the risks.

In such a highly uncertain environment, the interpretability of models is of utmost importance. Warren Buffett said, "Risk comes from not knowing what you are doing" [18]. As his word implies, investors may desire the model to be explainable to understand what they are doing. In fact, historically, investors and researchers have preferred linear factor models to explain asset prices (e.g., [15, 16]), which are often believed as interpretable. On the other hand, for machine learning-based strategies, the lack of interpretability can be an obstacle to practical use, without which investors cannot understand their own investments nor ensure accountability to customers. We believe that using the combination of simple formulae mentioned in the previous subsection can open up a possibility of interpretable machine learning-based trading strategies.

1.3 Our Contributions

To address the aforementioned challenges, we propose the Trader-Company method, a new metaheuristics-based method for stock price prediction. Our method is inspired by the role of financial institutions in the real-world stock markets. Figure 1 depicts the entire framework of our method. Our method consists of two main ingredients, *Traders* and *Companies*. A single Trader predicts the returns based on simple mathematical formulae, which are postulated to be good candidates for interpretable alpha factors. Brought together by a Company, Traders act as weak learners that provide partial information helping the Company's eventual prediction. The Company also updates the collection of Traders by generating (i.e., hiring) new candidates of good Traders as well as by deleting (i.e., dismissing) poorly performing Traders. This framework allows us to effectively search over the space of mathematical formulae having categorical parameters.

We demonstrate the effectiveness of our method by experiments on real market data. We show that our method outperforms several standard baseline methods in realistic settings. Moreover, we show that our method can find formulae that are profitable by themselves and simple enough to be interpreted by real-world investors.

2 PROBLEM DEFINITION

In this section, we provide several mathematical definitions of financial concepts and formulate our problem setting. Table 1 summarizes the notation we use in this paper.

2.1 Problem Setting

Our problem is to forecast future returns of stocks based on their historical observations. To be precise, let $X_i[t]$ be the price of stock i at time t, where $1 \leq i \leq S$ is the index of given stocks and $0 \leq t \leq T$ is the time index. Throughout this paper, we consider the logarithmic returns of stock prices as input features of models. That is, we denote the one period ahead return of stock i by

$$r_i[t] := \log(X_i[t]/X_i[t-1]) \approx \frac{X_i[t] - X_i[t-1]}{X_i[t-1]}. \tag{1}$$

We denote returns over multiple periods and returns over multiple periods and multiple stocks by

$$\boldsymbol{r}_i[u:v] = (r_i[u], \cdots, r_i[v]), \boldsymbol{r}_{i:j}[u:v] = (r_i[u:v], \cdots, r_j[u:v]) \tag{2}$$

Our main problem is formulated as follows.

Problem 1 (one-period-ahead prediction)**.** The predictor sequentially observes the returns $r_i[t]$ $(1 \leq i \leq S)$ at every time $0 \leq t \leq T$. For each time t, the predictor predicts the one-period-ahead return $r_i[t+1]$ based on the past t returns $\boldsymbol{r}_{1:S}[0:t]$. That is, the predictor's output can be written as

$$\hat{r}_i[t+1] = f_t(\boldsymbol{r}_{1:S}[0:t]) \tag{3}$$

for some function f_t that does not depend on the values of $\boldsymbol{r}_{1:S}[t:T]$.

Table 1: Notation.

Notation	Meaning	Def.
$X_i[t]$	stock price of stock i at time t where $1 \le i \le S, 0 \le t \le T$	§ 2.1
$r_i[t]$	logarithmic return of i at t	(1)
$\boldsymbol{r}_i[u:v]$	$(r_i[u], \cdots, r_i[v])$	(2)
$\boldsymbol{r}_{i:j}[u:v]$	$(r_i[u:v], \cdots, r_j[u:v])$	(2)
$\hat{r}_i[t], \hat{b}_i[t]$	predicted value of $r_i[t]$ and its sign	(3)(4)
M, P_j, Q_j, D_j F_j, A_j, O_j	hyper-parameters of Traders	(5)

2.2 Evaluation

To evaluate the goodness of the prediction, we use the cumulative return defined as follows. Given a predictor's output $\hat{r}_i[t+1]$, we define the "canonical" trading strategy as

$$\hat{b}_i[t+1] = \text{sign}(\hat{r}_i[t+1]). \tag{4}$$

Here, the value of $\hat{b}_i[t]$ represents the trade of stock i at time t. That is, $\hat{b}_i[t] = \pm 1$ respectively means that the strategy buys/sells one stock and sell/buy it back after one periods. Thus, equation (4) can be interpreted as a strategy that buys a unit amount of a stock if its one period ahead return is expected to be positive and otherwise sells it. Then, we define the cumulative return of the prediction $\hat{r}_i[t+1]$ as

$$C_i[t] = \sum_{u=0}^{t} \hat{b}_i[u+1] r_i[u+1] = \sum_{u=0}^{t} \text{sign}(\hat{r}_i[u+1]) r_i[u+1].$$

If the predictor could perfectly predict the sign of the one period ahead returns, the above canonical strategy yields the maximum cumulative return among all possible strategies that can only trade unit amounts of stocks. As such, we consider $C_i[t]$ as an evaluation major of the prediction.

3 TRADER-COMPANY METHOD

In this section, we present the *Trader-Company method*, a new metaheuristics-based prediction algorithm for stock prices.

Figure 1 outlines our proposed method. Our method consists of two main components, *Traders* and *Companies*, which are inspired by the role of human traders and financial institutes, respectively. A Trader predicts the returns using a simple model expressing realistic trading strategies. A Company combines suggestions from multiple Traders into a single prediction. To train the parameters of the proposed system, we employ an evolutionary algorithm that mimics the role of financial institutes as employers of traders. During training, a Company generates promising new candidates of Traders and deletes poorly performing ones. Below, we provide more detailed definitions and training algorithms for Traders and Company.

3.1 Traders - Simple Prediction Module

First, we introduce the Traders, which are the minimal components in our proposed framework. As mentioned in the introduction, trading strategies used in real-life trading are made of simple formulae involving a small number of arithmetic operations on the return values [26]. We postulate that there are a number of unexplored profitable market rules that can be represented by simple formulae, which leads us to the following definition of a parametrized family of formulae.

Definition 1. A Trader is a predictor of one period ahead returns defined as follows. Let M be the number of terms in the prediction formula. For each $1 \le j \le M$, we define P_j, Q_j as the indices of the stock to use, D_j, F_j as the delay parameters, O_j as the binary operator, A_j as the activation function, and w_j as the weight of the j-th term. Then, the Trader predicts the return value $r_i[t+1]$ at time $t+1$ by the formula

$$f_\Theta(\boldsymbol{r}_{1:S}[0:t]) = \sum_{j=1}^{M} w_j A_j(O_j(r_{P_j}[t-D_j], r_{Q_j}[t-F_j])). \tag{5}$$

where Θ is the parameters of the Trader:

$$\Theta := \{M, \{P_j, Q_j, D_j, F_j, O_j, A_j, w_j\}_{j=1}^{M}\}.$$

For activation functions A_j, we use standard activation functions used in deep learning such as the identity function, hyperbolic tangent function, hyperbolic sine function, and Rectified Linear Unit (ReLU). For the binary operators O_j, we use several arithmetic binary operators (e.g., $x+y$, $x-y$, and $x \times y$), the coordinate projection, $(x, y) \mapsto x$, the max/min functions, and the comparison function $(x > y) = \text{sign}(x-y)$.

Our definition of the Trader has several advantages. First, the formula (5) is ready to be interpreted in the sense that it has a similar form to typical human-generated trading strategies [26]. Second, the Trader model has a sufficient expressive power. The Trader has various binary operators as fundamental units, which allows it to represent any binary operations commonly used in practical trading strategies. Besides, the model also encompasses the linear models since we can choose the projection operator $(x, y) \mapsto x$ as O_j.

Ideally, we want to optimize the Traders by maximizing the cumulative returns:

$$\begin{aligned} \Theta^* &\in \arg\max_{\Theta} \textstyle\sum_u \text{sign}(f_\Theta(\boldsymbol{r}_{1:S}[0:u])) \cdot r_i[u+1] \\ &:= \arg\max_{\Theta} R(f_\Theta, \boldsymbol{r}_{1:S}[0:t], r_i[0:t+1]) \end{aligned} \tag{6}$$

However, it is difficult to apply common optimization methods since the objective in the right-hand side is neither differentiable nor continuous w.r.t. the parameter Θ. Therefore, we introduce a novel evolutionary algorithm driven by Company models, which we will describe below.

3.2 Companies - Optimization and Aggregation Module

As mentioned in the introduction, the behaviour of the financial market is highly unstable and uncertain, and thereby any single explanatory model is merely partially correct and transient. To overcome this issue and robustify the prediction, we develop a method to combine predictions of multiple Traders. This is in the same spirit as the general and long-standing framework of ensemble methods (e.g., [21] or Chapter 4 of [39]), but introducing an "inductive bias" that takes into account the dynamics of real-world financial markets would improve the performance of combined prediction. In

particular, given the fact that the stock prices are determined as a result of diverse investments made by institutional traders, it is reasonable to consider a model imitating the environments in which institutional traders are involved (i.e., financial institutions).

Algorithm 1 Prediction algorithm of Company

Input: $\boldsymbol{r}_{1:S}[0:t]$:stock returns before t, Traders :$\{\Theta_n\}_{n=1}^N$
Output: $\hat{r}_i[t+1]$: predicted return of stock i at t

function CompanyPrediction
  for $n = 1, \cdots, N$ **do**
    $P_n \Leftarrow f_{\Theta_n}(\boldsymbol{r}_{1:S}[0:t])$ ▷ Prediction by Trader (5)
  end for
  return Aggregate$(P_1, \cdots, P_N)$ ▷ Aggregation
end function

Algorithm 2 Educate algorithm of Company

Input: $\boldsymbol{r}_{1:S}[0:t]$:stock returns before t
Input: Traders. N : the number of Traders. Q: ratio.
Output: Traders

function CompanyEducate
  $R_n \Leftarrow R(f_{\Theta_n}, \boldsymbol{r}_{1:S}[0:t], r_i[0:t+1])$ ▷ Trader's return (6)
  $R^* \Leftarrow$ bottom Q percentile of $\{R_n\}$
  for $n \in \{m | R_m \leq R^*\}$ **do** ▷ for all bad traders
    Update w_i in (5) by least squares method
  end for
  return Traders
end function

Algorithm 3 Prune-and-Generate algorithm of Company

Input: $\boldsymbol{r}_{1:S}[0:t]$:stock returns before t, F: # of fit times
Input: N: the number of Predictors. Q: ratio.
Output: N' Predictors

$\Theta_n \sim$ Unifrom Distribution
for $k = 1, \cdots, F$ **do**
  $R_n \Leftarrow R(f_{\Theta}, \boldsymbol{r}_{1:S}[0:t], r_i[0:t+1])$ ▷ Trader's return (6)
  $R^* \Leftarrow$ bottom Q-percentile of $\{R_n\}$
  $\{\Theta_j\}_j \Leftarrow \{\Theta_n | R_n \geq R^*\}$ ▷ Pruning
  $\{\Theta_j\}_{j=1}^{N'} \sim$ GM fitted to $\{\Theta_j\}_j$ * ▷ Generation
end for
return N' Predictors with $\{\Theta_j\}_{j=1}^{N'}$

* If the parameter is an integer, we round it off.

In our framework, a Company maintains N Traders that act as weak learners or feature extractors, and aggregate them. Given N Traders specified by parameters $\Theta_1, \ldots, \Theta_n$ and the past observations of stock returns $\boldsymbol{r}_{1:S}[0:t]$, a Company predicts the future returns by

$$\hat{r}[t+1] = \text{Aggregate}(f_{\Theta_1}, \ldots, f_{\Theta_n}).$$

For clarity, this procedure is presented in Algorithm 1. Here, Aggregate can be an arbitrary aggregation function and allowed to have extra parameters. For example, we can use the simple averaging $\frac{1}{N}\sum_{n=1}^N f_{\Theta_n}(\boldsymbol{r}_{1:S}[0:t])$, linear regression or general trainable prediction models (e.g., neural networks and the Random Forest) that take the Traders' suggestions as the input features.

In order to achieve low training errors whilst avoiding overfitting, the Company should maintain the average quality as well as the diversity of the Traders' suggestions. To this end, we introduce the Educate algorithm (Algorithm 2) and the Prune-and-Generate algorithm (Algorithm 3), which update weights and formulae of Traders, respectively.

- **Educating Traders**: Recall that a single Trader (5) is a linear combination of M mathematical formula. A single Trader can perform poorly in terms of cumulative returns. However, if the Trader has good candidates of formulae (i.e., alpha factors), slightly updating the weights $\{w_j\}$ while keeping the formulae would significantly improve the performance. Algorithm 2 corrects the weights $\{w_j\}$ of the Traders achieving relatively low cumulative returns. Here, we update the weights by the least-squares method, which is solved analytically.
- **Pruning poorly performing Traders and generating new candidate good Traders**: If a Trader holds "bad" candidates of formulae, keeping that Trader makes no improvement on the prediction performance while it can increase risk exposures. In that case, it may be beneficial to simply remove that Trader and replace it with a new promising candidate. Algorithm 3 implements this idea. First, we evaluate the cumulative returns of the current set of Traders, and remove the Traders having relatively low returns. Then, we generate new Traders by randomly fluctuating the existing Traders with good performances. To this end, we fit some probability distribution to the current set of parameters, and draw new parameters from it. While the parameter specifying the formulae contain discrete variables such as indices of stocks and choices of arithmetic operations, we empirically found that fitting the Gaussian mixture distribution (i.e., a continuous multi-modal distribution) to discrete indices and discretizing the generated samples can achieve reasonably good performances. See Section 4 for detailed experimental results.

Using the above algorithms together, we can effectively search the complicated parameter space of Traders. Educate algorithm (Algorithm 2) is intended to be applied before pruning (Algorithm 3) to prevent potentially useful alpha factors from being pruned. In practice, given past observations of returns $\boldsymbol{r}_{1:S}[t_1:t_2]$, we can train the model by the following workflow.

(1) Educate a fixed proportion of poorly performing Traders by Algorithm 2.
(2) Replace a fixed proportion of poorly performing Traders with random new Traders by Algorithm 3.
(3) If the aggregation function Aggregate has trainable parameters, update them using the data $\boldsymbol{r}_{1:S}[t_1:t_2]$ and any optimization algorithm.
(4) Predict future returns by Algorithm 1.

We comment on some intuitions about the advantages of our method, although there is no theoretical guarantee in practical settings. Algorithm 3 increases the diversity of Traders by injecting

random fluctuations to existing good Traders. From a generalization perspective, injecting randomness may help to avoid overfitting to the current transient state of the market. From an optimization perspective, we can view our algorithm as a variant of evolutionary algorithms such as the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [19]. While the original CMA-ES generates new particles from a Gaussian distribution, we see that using a multi-modal distribution is practically important. See Section 4 for an empirical verification.

4 EXPERIMENTS

We conducted experiments to evaluate the performance of our proposed method on real market data. We compare our method with several benchmarks including simple linear models and state-of-the-art deep learning methods, and confirm the superiority of our method. We also demonstrate that our method can find profitable formulae (alpha factors) that are simple enough to interpret.

4.1 Datasets

Throughout the experiments, we used two real market datasets: (i) US stock prices listed on Standard & Poor's 500 (S&P 500) Stock Index and (ii) UK stock prices from the London Stock Exchange (LSE). S&P 500 and LSE have one of the highest trading volumes and market capitalization in the world. From a reproducibility viewpoint, we used data that were distributed online free from Dukascopy's Online data feed[1]. For the S&P 500 data, we used daily data for all the 500 stocks listed S&P 500 Stock Index in the period from May 19, 2000 to May 19, 2020. For the LSE data, we used hourly data for all the 77 stocks prices available on Dukascopy in the period from September 07, 2016 to September 07, 2019.

4.2 Evaluation Protocols

4.2.1 Time windows and execution lags. In our experiments, we employed two practical constraints, time windows and execution lags. In practice, we cannot use all the past observations of stock prices due to the time and space complexity. Also, we cannot trade stocks immediately after the observation of returns because we need some time to make an inference by the model and execute the trade. Thus, it is reasonable to introduce a time window $w > 0$ and an execution lag $l > 0$, so we train models using observations $\boldsymbol{r}_{i:S}[t-l-w : t-l]$ and predict returns at time $t+1$. Throughout experiments, we used $w = 10$ and $l = 1$.

4.2.2 Metrics. To evaluate the performances of prediction algorithms, we adopted three metrics defined as follows. For each metric, higher value is better. Let $r_i[t]$ be the return of stock i ($i \in \{1, \ldots, S\}$) at time t, and let $\hat{r}_i[t]$ be its prediction obtained from an arbitrary method. Recall $\hat{b}_i[t] = \mathrm{sign}(\hat{r}_i[t])$.

- **Accuracy (ACC)** the accuracy rate of prediction of the rise and drop of stock prices. $\mathrm{ACC}_i = \mathrm{P}[\mathrm{sign}(r_i[t]) = \hat{r}_i[t]]$.
- **Annualized Return(AR)**: Given predictions of the returns of stock i, the cumulative return is given as $C_i[t] := \sum_{u=0}^{t} \hat{b}_i[u+1] r_i[u+1]$. We define the Annualized Return **(AR)** averaged over all stocks as $\mathrm{AR} := 100 \times \frac{T_\mathrm{Y}}{T} \frac{1}{S} \sum_{i=1}^{S} C_i[T]$, where T_Y is the average number of periods contained in one year.
- **Sharpe Ratio (SR)**: The Sharpe ratio [36], or the Return/Risk ratio (R/R) is the return value adjusted by its standard deviation. That is, letting $\mu_i := \frac{1}{T} C_i[T] = \frac{1}{T} \sum_{t=1}^{T} \hat{b}_i[u+1] r_i[u+1]$ and$\sigma_i^2 := \frac{1}{T} \sum_{t=1}^{T} (\hat{b}_i[u+1] r_i[u+1] - \mu_i)^2$, and we define $\mathrm{SR}_i := \mu_i / \sigma_i$. Then, we report the average $\mathrm{SR} = \frac{1}{S} \sum_{i=1}^{S} \mathrm{SR}_i$.
- **Calmar Ratio (CR)**: We also use the Calmar ratio [43], another definition of adjuster returns. Define the Maximum DrawDown (MDD) as

$$\mathrm{MDD}_i := \max_{1 \le t \le T} \max_{t < s \le T} \left(1 - \frac{C_i[t]}{C_i[s]}\right).$$

  The Calmar ratio is defined as $\mathrm{CR}_i := \mathrm{AR}_i / \mathrm{MDD}_i$. In our experiments, we report $\mathrm{CR} = \frac{1}{S} \sum_{i=1}^{S} \mathrm{CR}_i$. Note that while both SR and CR are adjusted returns by its risk measures, CR is more sensitive to drawdown events that occur less frequently (e.g., financial crises).

4.3 Baseline Methods

We compared our methods with the following baseline methods:

- Market: a uniform Buy-And-Hold strategy.
- Vector Autoregression (VAR): a commonly-used linear model for financial time series forecasting [37].
- Random Forest (RF): a commonly-used ensemble method [5].
- Multi Head Attention (MHA): a deep learning algorithm for time series prediction [40].
- Long- and Short-Term Networks (LSTNet): a deep-learning-based algorithm which combines Convolutional and Recurrent Neural Network [27].
- State-Frequency Memory Recurrent Neural Networks (SFM)[2]: a deep learning-based stock price prediction algorithm that incorporates the concept of Fourier Transform into Long Short-Term Memory [44].
- Symbolic Regression by Genetic Programming (GP): a predicton algorithm using genetic programming [33].

To verify the effects of individual technical components in our proposed method (TC), we also compared the following "ablation" models.

- Changing the Trader structure: **TC linear** only uses the linear activation function $x \mapsto x$, so the eventual prediction of a Campany becomes just a linear combination of several binary operations. **TC unary** only uses the unary operator $O(x, y) = x$.
- Changing the optimization algorithm: **TC w/o educate** does not execute the Educate Algorithm (Algorithm 2), so Traders can be discarded even if they have promising formulae. **TC w/o prune** does not execute the pruning step in Algorithm 3, so a Company keeps poorly performing Traders. **TC unimodal** uses the Gaussian distributions instead of the Gaussian mixtures in the generation step. **TC MSE** uses the mean squared loss as scores for educating and pruning, so it does not use the cumulative returns at all.

Table 2 lists the hyper-parameters used in the baseline algorithms.

[1] https://www.dukascopy.com/

[2] There was an unintended data leak in the implementation published by the author. Therefore, we fixed it in our experiment for fairness.

Table 2: Hyper-parameters in Experiments

Parameter	Value	Definition
M	$\{1, \cdots, 10\}$	(5)
D_j, F_j	$\{0, \cdots, 10\}$	(5)
$A_j(x)$	$\{x, \tanh(x), \exp(x), \text{sign}(x), \text{ReLU}(x)\}$	(5)
$O_j(x, y)$	$\{x+y, x-y, xy, x, y, \max(x, y), \min(x, y), x > y, x < y, \text{Corr}(x, y)\}$	(5)
N	100	Algorithm 1
Aggregate	Linear regression	Algorithm 1
Q	0.5	Algorithm 3

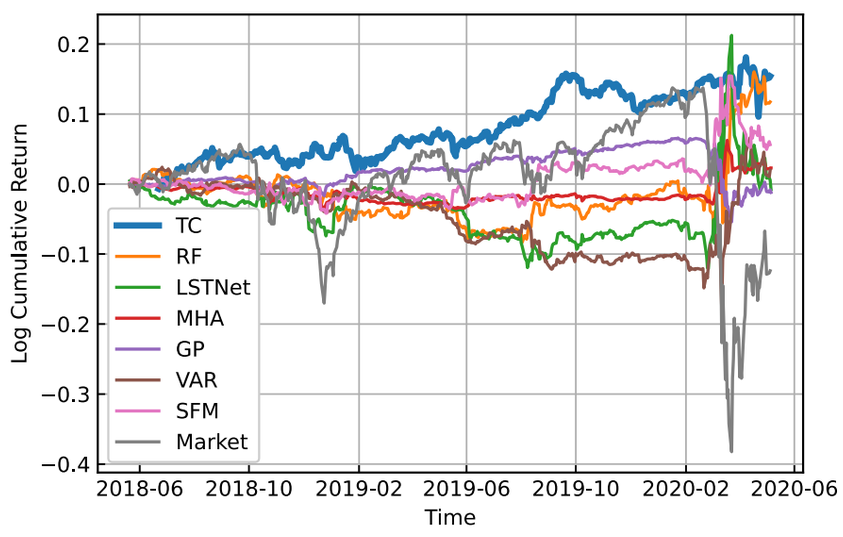

Figure 2: Cumulative returns on US market

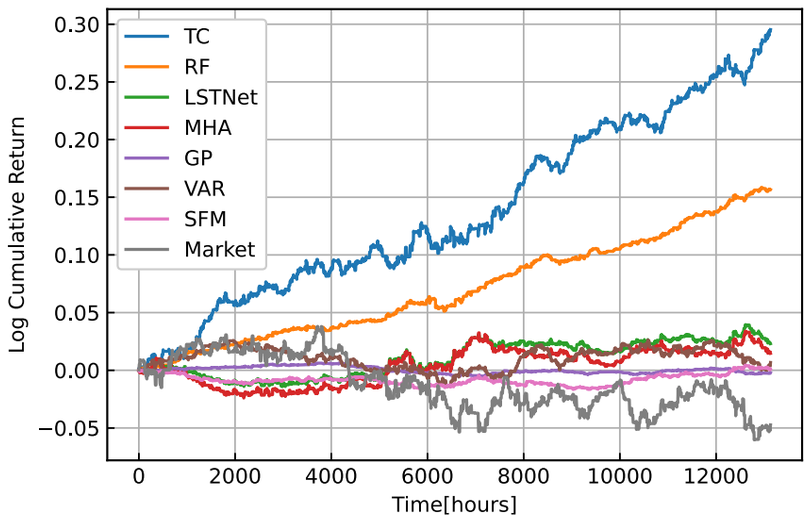

Figure 3: Cumulative returns on UK market

4.4 Performance Evaluation on Real Market Data

4.4.1 Offline prediction. First, we trained the models using the first half of the datasets, and then evaluated the performances of the models with frozen parameters using the latter half. For US market, we used the data before May 2018 for training and the rest for testing. For UK market, we used the first one and a half years for training and the rest for testing.

Table 3 and Table 4 show the comparisons of between our proposed method (TC) and other baseline methods on US and UK markets, respectively. All methods are evaluated using three evaluation metrics (AR, SR, and CR). For methods depending on random initializations, we run the evaluations for each method 100 times

Table 3: Performance comparison on US markets

	ACC(%)	AR(%)	SR	CR
Market	55.09	-5.45	-0.2	-0.12
VAR	49.86	3.51	0.28	0.24
MHA	48.16 ± 1.1	0.66 ± 0.54	0.12 ± 0.10	0.11 ± 0.09
LSTNet	47.51 ± 1.12	3.61 ± 2.09	0.18 ± 0.10	0.16 ± 0.1
SFM	50.14 ± 1.09	5.95 ± 1.16	0.52 ± 0.10	0.50 ± 0.18
GP	54.99 ± 1.15	-0.27 ± 0.67	-0.03 ± 0.10	-0.02 ± 0.08
RF	53.74 ± 1.11	4.38 ± 1.60	0.28 ± 0.11	0.25 ± 0.13
TC linear	53.23 ± 1.18	2.81 ± 0.36	0.86 ± 0.10	0.93 ± 0.28
TC unary	51.14 ± 1.17	1.50 ± 0.33	0.46 ± 0.11	0.44 ± 0.17
TC w/o educate	52.36 ± 1.15	3.69 ± 0.32	1.12 ± 0.10	1.36 ± 0.38
TC w/o prune	50.31 ± 1.14	1.08 ± 0.33	0.34 ± 0.11	0.30 ± 0.13
TC unimodal	50.33 ± 1.17	0.08 ± 0.35	0.02 ± 0.11	0.02 ± 0.08
TC MSE	51.35 ± 1.19	1.56 ± 0.54	0.30 ± 0.11	0.26 ± 0.13
TC (Proposed)	**55.68 ± 1.16**	**10.72 ± 0.86**	**1.32 ± 0.11**	**1.57 ± 0.44**

Table 4: Performance comparison on UK markets

	ACC(%)	AR(%)	R/R	CR
Market	50.009	-3.34	-0.177	-0.076
VAR	49.965	0.54	0.064	0.031
MHA	49.943 ± 0.991	1.01 ± 2.91	0.04 ± 0.15	0.03 ± 0.05
LSTNet	49.997 ± 1.007	1.53 ± 3.29	0.07 ± 0.18	0.05 ± 0.08
SFM	50.472 ± 8.810	0.23 ± 4.23	0.00 ± 0.22	0.02 ± 0.10
GP	50.017 ± 2.128	-0.09 ± 1.47	-0.02 ± 0.36	0.05 ± 0.12
RF	50.719 ± 1.177	10.45 ± 1.98	1.23 ± 0.23	1.17 ± 0.56
TC	**50.928 ± 1.115**	**32.32 ± 1.04**	**2.23 ± 0.07**	**3.32 ± 0.44**

with different random seeds and provide the means and the standard deviations. Also, Figure 2 and Figure 3 show the cumulative returns on US and UK markets, respectively.

Overall, our method outperformed the other baselines in the presented three evaluation metrics. Some interesting observations are as follows.

Importance of Traders: Comparing several baseline methods and ablation models, we found that the structure of Traders is of crucial importance.

First, in the definition of Traders (5), we restrict the formulae to those represented by the binary operators O_j. This means that Traders rule out formulae that leverage interactions between three or more terms, which can reduce the complexity of the entire model without losing the expressive power. This is corroborated by the following observations: Among the baseline methods, a simple linear method (VAR) achieved a relatively good performance. VAR estimates its coefficients by the ordinary least-squares method, which means that the prediction of the return of each individual stock can be a "dense" linear combination of past observations. On the other hand, TC linear, which improved SR significantly upon VAR, finds the solution among linear combinations of features made of at most two observations. On the other hand, we can also see that using only unary operations (TC unary) greatly deteriorates the performance.

Second, comparing TC and TC linear, we see that introducing non-linear activation functions also improves the performance.

Table 5: Prediction formula extracted from the best Traders. These expressions are for predicting the returns at $t+1$

# of terms	LLOY_{t+2}
1	$= -2.28(\mathrm{SHP}_t > \mathrm{AHT}_{t-3})$
2	$= -0.80\ \mathrm{sign}(\mathrm{WPP}_{t-3} + \mathrm{SKY}_t)$ $-1.85\ \mathrm{sign}(\min(\mathrm{AV}_{t-2}, \mathrm{SHP}_{t-5}))$
3	$= 4.919\mathrm{ReLU}(\mathrm{AHT}_{t-3})$ $+5.859\ \mathrm{sign}(\max(\mathrm{WEIR}_{t-2}, \mathrm{WTB}_{t-5}))$ $-1.07(\mathrm{PFC}_{t-1} > \mathrm{DGE}_{t-3})$

Third, TC also outperformed another off-the-shelf ensemble method (RF). RF combines non-linear predictors given by decision trees, i.e., indicator functions of rectangles (see e.g., Chapter 9 of [21]). RF requires many decision trees to approximate binary operations such as $\max x, y$ or $x > y$. Hence, when these operations are actually important for constructing alpha factors, RF can increase the redundancy and the model complexity, which leads to poor performance especially in SR and CR. In fact, these operations frequently appear in good Traders (Table 5).

Importance of combining multiple formulae: Among the baseline methods, GP outputs a single mathematical formula by using genetic programming [33]. However, this did not work well in our experiments in which we adopted reasonably long test periods. This may reflect the fact that any single formula is ephemeral due to the (near) efficiency of the markets. On the other hand, our method that maintains multiple formulae performed well over the test period.

Importance of optimization heuristics: We found that each individual optimization technique presented in Section 3.2 significantly improves the performance. First, the pruning step seems quite important (cf. TC w/o prune). Regarding the scores for the pruning, using the MSE instead of the cumulative returns deteriorates the performance (cf. TC MSE). Second, we can see that introducing the education step also improves the overall performance (cf. TC w/o educate). Otherwise Companies may discard Traders that have possibly good formulae. Lastly, using multimodal distribution in the generation step (Algorithm 3) is quite important. If we instead use a unimodal distribution (cf. TC unimodal), the performance is substantially deteriorated. A possible reason is that a unimodal distribution concentrates around the means of discrete indices, which does not make sense.

Comparison to other non-linear predictors: We also compared our method to other complex non-linear models. MHA, LSTNet and SFM are prediction methods based on deep learning. Among these, SFM performed relatively well in US market, but none of them achieved good performances in UK market. Our method consistently outperformed these methods.

4.4.2 Online prediction. In the previous experiment, we adopted a simple train/test splitting. However, in practice, it is not reasonable to use a single model for a long time, and we might update the model more frequently to follow structural changes of the markets. Here, using the US data, we also evaluated our method in a sequential prediction setting. We sequentially updated the models every year, where we used all the past observations for training. Figure 4 shows the cumulative returns on US market. Our method performed well also in this setting.

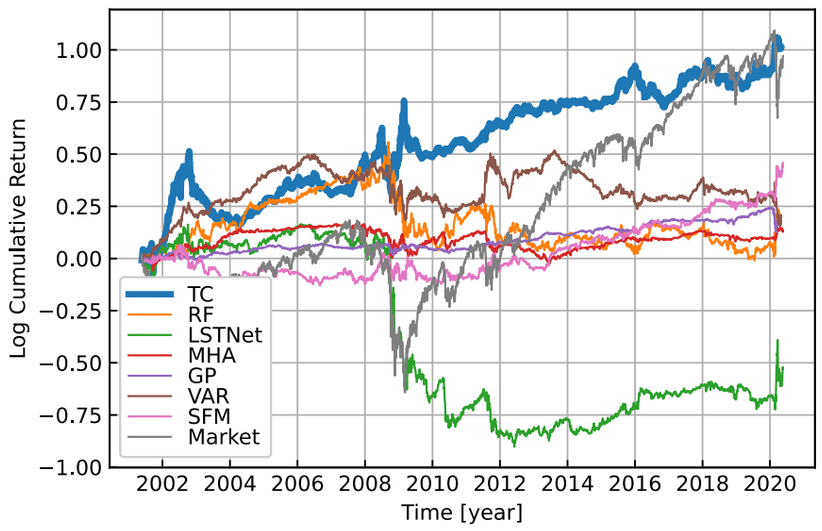

Figure 4: Cumulative returns on US market in the online prediction setting

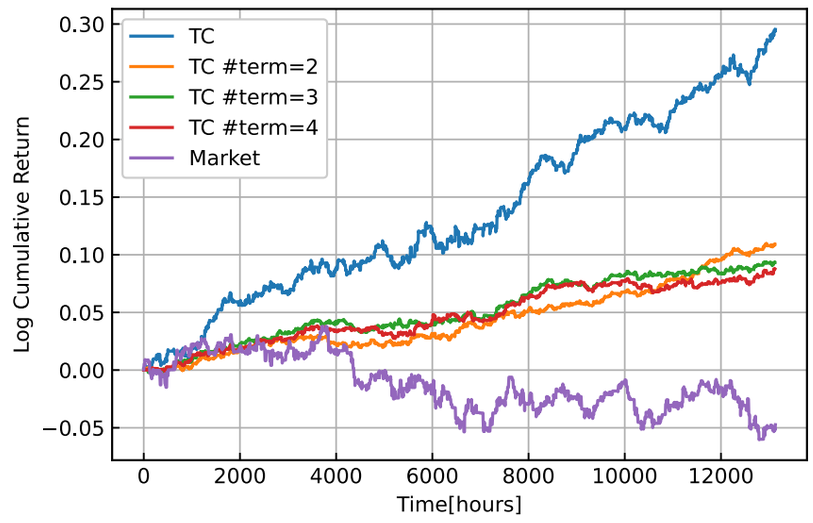

Figure 5: Cumulative returns achieved by individual Traders extracted from Companies. "TC" is the overall performance of our method. "TC # term = M" stands for a single best performing Trader extracted from a Company with fixed parameter M.

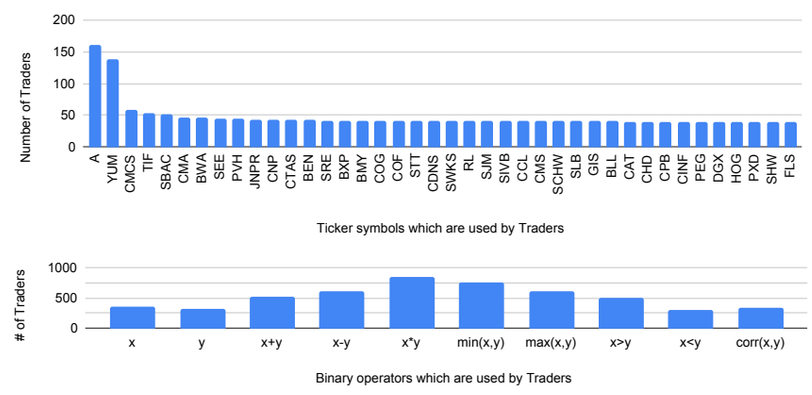

Figure 6: The number of times each parameter was used.(Upper) indices P_j, Q_j. (Lower) operators O_j

4.5 On Interpretablity of Traders

So far, we have evaluated the overall performance of the proposed method. Here, we investigate the performance of a single Trader or a formula extracted from the trained model, and discuss about the interpretation of obtained formulae.

First, we investigate the performance of a single *Trader*. Recall that a Trader (5) is given as a linear combination of M mathematical formulae. Thus, the expressive power of a single Trader increases as M increases. Here, using the UK data, we trained the Companies by restricting M to be a fixed value in $\{1, 2, 3, 4\}$ (Note that, in our method, M for each Trader is trainable by default). Then, we extracted the best performing Trader from each trained Company, and evaluated the performance for the test period. Figure 5 shows the result. We can see that our method can find Traders that achieve positive returns by themselves, while the total return of the overall market is negative during the test period. If we increase the number of terms M, the cumulative return of a single Trader at the end of the test period decreased (albeit slightly), which suggests that increasing the expressive powers of individual Traders is prone to overfitted formulae and does not necessarily result in a single profitable formula. Meanwhile, we should note that the overall performance of the Company (TC in Figure 5) is much better than a single Trader.

Next, we consider the interpretability issue. Table 5 lists the actual formulae extracted from the trained Traders. Regarding the meanings of the ticker symbols used in the formulae, see the website of London Stock Exchange [3]. For example, LLOY_{t+2} indicates the return from $t+1$ to $t+2$ of Lloyds Bank. We believe that each formula would be informative for real-world investors and can provide useful insights into their investments. Notably, we found that the extracted formulae often contain binary operations such as $\min(x, y)$, $\max(x, y)$ and pairwise comparisons ($x > y$). These operators are difficult to be approximated by decision trees that can only represent rectangular regions, which may be the reason for the superiority of our method over the Random Forest.

Figure 6 shows which ticker symbol and binary operator are used by Traders and how many times to predict the return of AAPL (Apple Inc.). From Figure 6, we can interpret which stocks the Company is paying attention to and binary operators the Traders are using.

5 RELATED WORK

5.1 Financial Modelling and Machine Learning

There are two established approaches to financial time series modeling. Firstly, many *statistical time series models* that aim to describe the generative processes of financial time series have been developed. For example, the autoregressive integrated moving average (ARIMA) model and Vector autoregression (VAR) are often used in financial time series prediction [4, 29]. Secondly, another direction is to use *factor models*, which aim to explain asset prices using not only the structure of each individual time series but also the cross-section information and other kinds of financial information. To name a few, Fama–French's three-factor model [15], Carhart four-factor model [7], and Fama-French's five-factor model [16] are among the most important ones. More than 300 identified factors are reported as of 2012 [20].

Recent financial econometrics is characterized by the combination of financial modeling and machine learning methods, especially deep learning. Although there are many research directions in this flourishing field, these include directions that (i) apply sophisticated time series models to financial problems and (ii) incorporate data of different modalities into the prediction. Sezer et al. conducted an extensive survey of these applications [35]. For the first direction, Zhang et al. [44] proposed to used frequency information to forecast stock prices. Lai et al. [27] proposed a method to extract long-term and short-term patterns by combining CNN and RNN. For the second direction, the use of news, social media and networks among companies is also active [8, 12, 23, 42].

However, it is folklore among experts that, under the transient and uncertain environments of financial markets, complex models (including neural networks) do not work well as expected, and traditional simpler models are more preferred. In partcular, Makridakis et al. [30] pointed out the superiority of "traditional" statistical models over machine learning models in financial time series. This motivates us to leverage simple units of models such as formulaic alphas [13, 26] and deal with the uncertainty by bootstrapping them, instead of using "black-box" deep learning-based methods.

5.2 Ensemble Methods

Combining multiple predictions is a long-standing approach in data science [21, 39]. Generally speaking, model selection and model ensemble are both important ideas (e.g., Chapter 8 of [21]). However, in some situations, selecting only a single model with the (temporal) best track record can lead to suboptimal performances, which has been confirmed empirically (e.g., Section 7.2 of [39]) and theoretically (e.g., [25]). Therefore, in many situations, ensemble-type methods might be the first candidate to try.

In ensemble methods, some techniques such as pruning and random generation of experts have shown to be effective in various situations. For example, it has been widely known that eliminating poorly performing experts (e.g., [17] and Section 7.3 of [39]) or partial structures of experts (e.g., [6]) can improve the overall performance. The idea of random generation of experts has been used in the Random Forest or the Random Fourier Features method [34]. We would like to stress that, as we demonstrated in Section 4, the designs of pruning and generation schemes are crucially important in financial time series prediction.

5.3 Metaheuristics in Finance

Over the years, many research have been done on the application of metaheuristics to finance [3]. Soler-Dominguez et al. [38] has done an extensive survey on these application. While portfolio optimization and index tracking and enhanced indexation are active applications of metaheuristics, the application of Genetic Programming (GP) is common for stock price prediction. Index prediction method, combination with self-organizing map (SOM), one using multi-gene and one using hybrid GP were proposed [2, 22, 31].

6 CONCLUSION

We proposed a new prediction method for financial time series. Our method consists of two main ingredients, the Traders and the Company. The Traders aim to predict the future returns of stocks by simple mathematical formulae, which can be naturally interpretable as "alpha factors" in finance literature. The Company aggregates

[3] https://www.londonstockexchange.com/

the predictions of Traders to overcome the highly uncertain environments of financial markets. The Company also provides a novel training algorithm inspired by real-world financial institutes, which allows us to search over the complicated parameter space of Traders and find promising mathematical formulae efficiently. We demonstrated the efficacy of our method through experiments on US and UK market data. In particular, our method outperformed some common baseline methods in both markets, and an ablation study showed that each individual technique in our proposed method does improve the overall performance. We focused on forecasting stock prices throughout this paper, and an interesting future direction is to investigate the applicability of our method to other types of assets.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive suggestions and comments. We also thank Masaya Abe, Shuhei Noma, Prabhat Nagarajan and Takuya Shimada for helpful discussions.

REFERENCES

[1] Steven Achelis. 2000. *Technical Analysis from A to Z, 2nd Edition.* McGraw-Hill, New York.

[2] Sara Elsir M. Ahmed, Alaa F. Sheta, and Hossam Faris. 2015. Evolving Stock Market Prediction Models Using Multigene Symbolic Regression Genetic Programming. *Artificial Intelligence and Machine Learning AIML* 15, 1 (June 2015), 11–20.

[3] Franklin Allen and Risto Karjalainen. 1999. Using genetic algorithms to find technical trading rules. *Journal of Financial Economics* 51, 2 (Feb. 1999), 245–271.

[4] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. 2015. *Time Series Analysis: Forecasting and Control (Wiley Series in Probability and Statistics).* Wiley, New York, NY.

[5] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.

[6] L. Breiman, J. Friedman, R Olshen, , and C. Stone. 1984. *Classification and Regression Trees.* Wadsworth.

[7] Mark M. Carhart. 1997. On Persistence in Mutual Fund Performance. *The Journal of Finance* 52, 1 (March 1997), 57–82.

[8] Yingmei Chen, Zhongyu Wei, and Xuanjing Huang. 2018. Incorporating Corporation Relationship via Graph Convolutional Neural Networks for Stock Price Prediction. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management - CIKM '18* (Torino, Italy). ACM Press, New York, NY, USA, 1655–1658.

[9] R. Cont. 2001. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance* 1, 2 (Feb. 2001), 223–236.

[10] Marcos López de Prado. 2015. The Future of Empirical Finance. *The Journal of Portfolio Management* 41, 4 (2015), 140–144.

[11] Marcos Lopez de Prado. 2018. *Advances in Financial Machine Learning.* Wiley, New York, NY.

[12] Xiao Ding, Yue Zhang, Ting Liu, and Junwen Duan. 2015. Deep Learning for Event-Driven Stock Prediction. In *Proceedings of the 24th International Conference on Artificial Intelligence (IJCAI'15).* AAAI Press, Buenos Aires, Argentina, 2327–2333.

[13] Igor Tulchinsky et al. 2015. *Finding Alphas: A Quantitative Approach to Building Trading Strategies.* Wiley.

[14] Eugene F. Fama. 1970. Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance* 25, 2 (May 1970), 383.

[15] Eugene F. Fama and Kenneth R. French. 1992. The Cross-Section of Expected Stock Returns. *The Journal of Finance* 47, 2 (June 1992), 427–465.

[16] Eugene F. Fama and Kenneth R. French. 2015. A five-factor asset pricing model. *Journal of Financial Economics* 116, 1 (April 2015), 1–22.

[17] Jerome H. Friedman. 1991. Multivariate Adaptive Regression Splines. *Annals of Statistics* 19, 1 (1991), 1–67.

[18] Robert Hagstrom. 1997. *The Warren Buffett way : investment strategies of the world's greatest investor.* J. Wiley, New York.

[19] Nikolaus Hansen and Andreas Ostermeier. 1996. Adapting arbitrary normal mutation distributions in evolution strategies: the covariance matrix adaptation. In *Proceedings of IEEE International Conference on Evolutionary Computation.* IEEE, Nagoya, Japan, 312–317.

[20] Campbell R. Harvey, Yan Liu, and Heqing Zhu. 2015. … and the Cross-Section of Expected Returns. *Review of Financial Studies* 29, 1 (Oct. 2015), 5–68.

[21] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning* (2nd edition ed.). Springer-Verlag.

[22] Chih-Ming Hsu. 2011. A hybrid procedure for stock price prediction by integrating self-organizing map and genetic programming. *Expert Syst. Appl.* 38 (may 2011), 14026–14036.

[23] Ziniu Hu, Weiqing Liu, Jiang Bian, Xuanzhe Liu, and Tie-Yan Liu. 2018. Listening to Chaotic Whispers. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining - WSDM '18.* Association for Computing Machinery, New York, NY, USA, 261–269.

[24] Narasimhan Jegadeesh and Sheridan Titman. 1993. Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency. *The Journal of Finance* 48, 1 (1993), 65–91.

[25] A. Juditsky, P. Rigollet, and A. B. Tsybakov. 2008. Learning by mirror averaging. *Annals of Statstics* 36, 5 (2008), 2183–2206.

[26] Zura Kakushadze and Juan Andrés Serur. 2018. *151 Trading Strategies.* Springer International Publishing, Cham, Switzerland.

[27] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. 2018. Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval* (Ann Arbor, MI, USA) *(SIGIR '18).* Association for Computing Machinery, New York, NY, USA, 95–104.

[28] Zhige Li, Derek Yang, Li Zhao, Jiang Bian, Tao Qin, and Tie-Yan Liu. 2019. Individualized Indicator for All: Stock-wise Technical Indicator Optimization with Stock Embedding. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining - KDD '19.* Association for Computing Machinery, New York, NY, USA, 894–902.

[29] Helmut Lütkepohl. 2005. *New Introduction to Multiple Time Series Analysis.* Springer Berlin Heidelberg, Berlin. https://doi.org/10.1007/978-3-540-27752-1

[30] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2018. Statistical and Machine Learning forecasting methods: Concerns and ways forward. *PLOS ONE* 13, 3 (March 2018), 1–26.

[31] Viktor Manahov, Robert Hudson, and Hafiz Hoque. 2015. Return predictability and the 'wisdom of crowds': Genetic Programming trading algorithms, the Marginal Trader Hypothesis and the Hayek Hypothesis. *Journal of International Financial Markets, Institutions and Money* 37 (July 2015), 85–98.

[32] R. David McLean and Jeffrey E. Pontiff. 2016. Does Academic Research Destroy Stock Return Predictability? *The Journal of Finance* 71, 1 (Jan. 2016), 5–32.

[33] Riccardo Poli, William B. Langdon, and Nicholas Freitag McPhee. 2008. *A Field Guide to Genetic Programming.* Lulu Enterprises, UK Ltd, Egham, United Kingdom.

[34] Ali Rahimi and Benjamin Recht. 2008. Random Features for Large-Scale Kernel Machines. In *Advances in Neural Information Processing Systems 20.* 1177–1184.

[35] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. 2019. Financial Time Series Forecasting with Deep Learning : A Systematic Literature Review: 2005-2019. arXiv:1911.13288 [cs.LG]

[36] William F Sharpe. 1964. Capital asset prices: A theory of market equilibrium under conditions of risk. *The journal of finance* 19, 3 (1964), 425–442.

[37] Christopher A. Sims. 1980. Macroeconomics and Reality. *Econometrica* 48, 1 (Jan. 1980), 1.

[38] Amparo Soler-Dominguez, Angel A. Juan, and Renatas Kizys. 2017. A Survey on Financial Applications of Metaheuristics. *Comput. Surveys* 50, 1 (April 2017), 1–23.

[39] Allan Timmermann. 2006. Forecast Combinations. In *Handbook of Economic Forecasting.* Elsevier, Amsterdam, Netherlands, 135–196.

[40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.). Curran Associates, Inc., Red Hook, NY, USA, 5998–6008.

[41] Thomas Wiecki, Andrew Campbell, Justin Lent, and Jessica Stauth. 2016. All That Glitters Is Not Gold: Comparing Backtest and Out-of-Sample Performance on a Large Cohort of Trading Algorithms. *The Journal of Investing* 25, 3 (Aug. 2016), 69–80.

[42] Yumo Xu and Shay B. Cohen. 2018. Stock Movement Prediction from Tweets and Historical Prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).* Association for Computational Linguistics, Melbourne, Australia, 1970–1979.

[43] Terry W Young. 1991. Calmar ratio: A smoother tool. *Futures* 20, 1 (1991), 40.

[44] Liheng Zhang, Charu Aggarwal, and Guo-Jun Qi. 2017. Stock Price Prediction via Discovering Multi-Frequency Trading Patterns. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '17* (Halifax, NS, Canada). Association for Computing Machinery, New York, NY, USA, 2141–2149.